

The Original Transformer Architecture

A Primer for Large Language Models

1. Setting the stage

The Large Language Models we use today, ChatGPT, Claude, Gemini, and others, are all built on top of a neural network architecture called the Transformer. Before we can talk meaningfully about LLMs and how to prompt them, it helps to understand, at a high level, what a Transformer is and why it was designed the way it was.

The Transformer was originally introduced in 2017 in a paper titled “Attention Is All You Need” (often shortened to AIAYN). The problem the authors were solving was machine translation: given a sentence in English, produce the same sentence in French. This may sound narrow, but the architecture they designed turned out to be remarkably general, and it is now the foundation of nearly every modern language model.

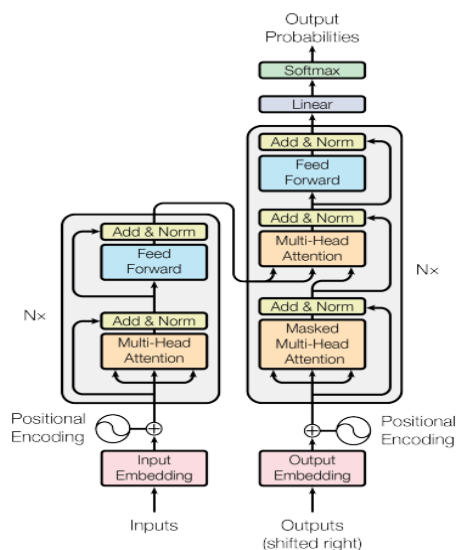


Image Source: The original research paper [Attention Is All You Need](#)

Translation is a useful lens because it tells us what the architecture needs to be good at. A translation model needs to do two distinct jobs: first, deeply understand the input sentence, and second, generate the output sentence word by word while staying faithful to the input. Keep this in mind, it is why the original Transformer has two halves, and why later models sometimes keep only one of them.

There is also a deeper challenge that language poses to any neural network: meaning often depends on words that sit far apart. Consider the sentence:

Anchor example: *“She poured water from the pitcher into the glass until it was full.”*

To understand what “it” refers to here, you need to look back across the whole sentence and weigh several clues “into the glass”, “until it was full”, the physics of pouring. A good language model needs the same flexibility: every word should be able to look at every other word and decide what is relevant. The Transformer was designed around exactly this idea. We will return to this sentence throughout the document as our anchor example.

2. From text to numbers

Neural networks do not operate on text directly; they operate on numbers. So before a sentence can enter a Transformer, it has to be converted into a numerical form. This happens in two steps.

Tokenization

The sentence is first split into smaller pieces called tokens. Tokens are usually whole words, but sometimes word fragments for longer or rarer words. Our anchor sentence might be split like this:

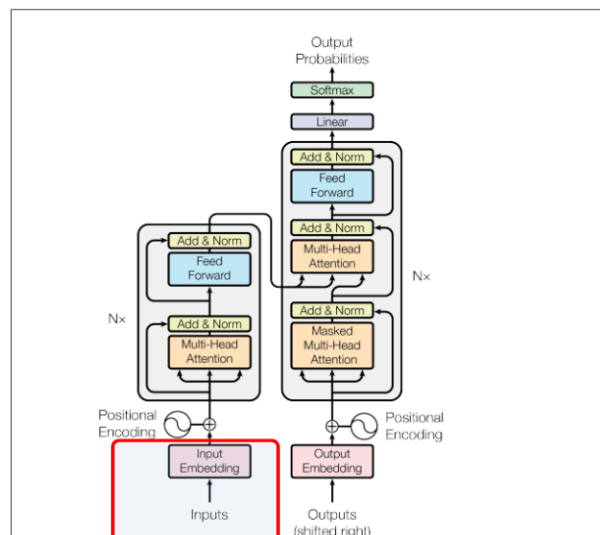
Tokens: [“She”, “poured”, “water”, “from”, “the”, “pitcher”, “into”, “the”, “glass”, “until”, “it”, “was”, “full”]

Each token is then mapped to an ID number from a fixed vocabulary that the model knows.

Embeddings

Each token ID is then replaced with a vector of numbers that captures the token’s meaning. Tokens with similar meanings end up with similar vectors. This vector is called an embedding. So the word “pitcher” becomes something like a list of several hundred numbers that represents what “pitcher” means to the model.

After this step, our sentence is no longer text. It is a sequence of vectors, one per token, and this is what flows into the rest of the network.



3. Giving the model a sense of order

There is a subtle problem with what we have so far. A Transformer is designed to look at all the words in a sentence at the same time, rather than one after another. That parallelism is a feature, not a bug; it is what makes Transformers fast to train and good at connecting distant words. But it comes with a cost: the model, by itself, has no idea what order the words were in.

Consider how differently these sentences behave:

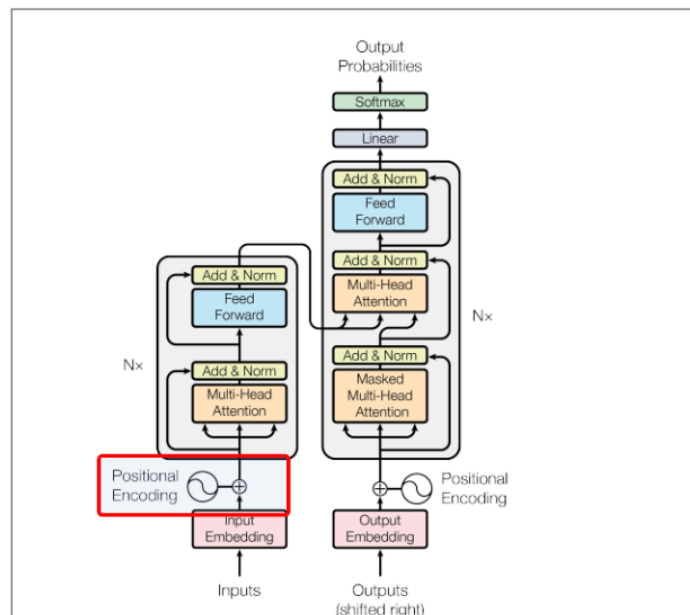
Compare: “She poured water from the pitcher into the glass.” vs. “She poured water from the glass into the pitcher.”

Same words, very different meanings. To a model that sees all tokens at once without order, these would look identical. Clearly, order matters.

Positional encoding

The fix is called positional encoding. Alongside the meaning vector for each token, we add a second set of numbers that tells the model where that token sits in the sentence first, second, third, and so on. These two are combined into a single vector per token that now carries both what the word is and where it is.

A useful analogy: imagine everyone in a group photo holding up their seat number. Each person is still themselves, but now you can also tell who was sitting where. The model now has everything it needs to reason about word order.



4. Attention: the heart of the Transformer

Attention is the single most important idea in the Transformer, and it is what gives the architecture its power. We will build it up in layers.

The intuition

When you read our anchor sentence, “She poured water from the pitcher into the glass until it was full,” your brain quietly figures out that “it” refers to the glass. It does this by weighing the other words in the sentence: “into the glass” matters a lot, “until” and “full” add supporting evidence, “she” and “poured” matter less for this particular question.

Attention is the model doing the same thing. For every word, the model decides how much every other word matters to it, and then builds a better representation of that word by pulling in information from the words that matter most.

Queries, keys, and values

Mechanically, the model does this using three vectors for each token, with names borrowed loosely from how search works:

- **Query** what this token is looking for.
- **Key** a label describing what this token offers.
- **Value** the actual information this token carries.

Think of it like a library. Your query is what you want to know. The keys are labels on the shelves. The values are the books themselves. You compare your query to every label, find the best matches, and take the corresponding books. Attention does this for every token, against every other token, all at once.

For our anchor sentence, when the token “it” issues its query (“what do I refer to?”), The keys for “glass”, “pitcher”, “into”, and “full” will all score as relevant. The values from those tokens get blended into “it”’s new representation, with “glass” contributing most.

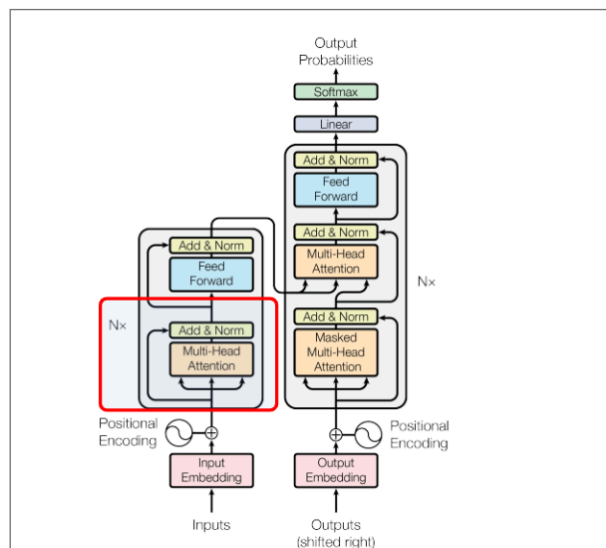
Self-attention

When every word in a sentence queries every other word in the same sentence, as in the example above, this is called self-attention. The sentence is, in effect, talking to itself, and every token ends up with a richer representation informed by its context.

Multi-head attention

A single attention operation can only capture one kind of relationship at a time. But language has many kinds of relationships running in parallel: grammar, meaning, reference, tone. The Transformer handles this by running several attention operations simultaneously, each called a head. One head might learn to track grammatical structure, another might track pronoun references, and another might track topic. Their outputs are then combined.

Analogy: several readers each highlight the same sentence for different reasons, one for grammar, one for meaning, one for references, and then pool their notes. The result is a much richer understanding than any single reader could produce alone. This is what “multi-head attention” means in practice.



5. The supporting cast

Attention is the star of the show, but a Transformer has a few more components that make attention actually work well in practice. These are the pieces of plumbing that sit around attention inside each layer of the network.

The feed-forward network

After attention has mixed information between words, each word's updated vector passes through a small neural network on its own. If attention is "talk to your neighbors and gather information," the feed-forward network is "now go think about what you heard." It lets each token independently process and refine the information it just picked up.

Residual connections

Before each sub-layer (like attention or the feed-forward network) modifies a token's vector, a copy of the original is saved and added back to the result. This ensures that important information is never lost, no matter how many layers the data passes through. Analogy: keeping a copy of your draft before every round of edits, so nothing important gets accidentally erased.

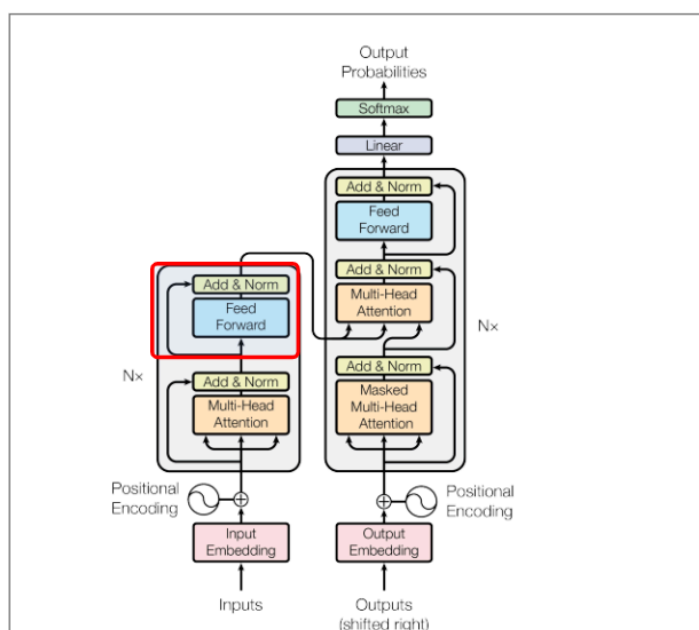
Layer normalization

As information flows through many layers, the numbers inside the network can drift, some getting very large, others very small, which makes training unstable. Layer normalization acts

like a volume knob that keeps the signals in a reasonable range at each step, so the network trains smoothly.

Stacking

A single Transformer layer attention + feed-forward + the plumbing above is powerful, but one layer alone is not enough. The real trick is to stack many such layers on top of each other. The original AIAYN Transformer stacked six. Modern LLMs stack dozens or even hundreds. Each layer refines the representation built by the one below it, and by the top of the stack, the model has built up a very deep understanding of the input.



6. Putting it together: the original encoder-decoder

We now have all the pieces we need to assemble the original Transformer. Recall the task it was designed for: translation. Taking an English sentence and producing the French equivalent, for example. This task has two distinct jobs, and the architecture has two distinct halves to match.

The encoder stack

The encoder's job is to read and understand the input sentence. It is a stack of identical layers, each containing self-attention, a feed-forward network, and the supporting plumbing from

Section 5. The input sentence flows in at the bottom as token embeddings (plus positional encodings), and by the time it reaches the top of the stack, each token's vector has been enriched with information from every other token in the sentence. The encoder's output is a rich numerical representation of the full input.

For our anchor example, the encoder's output would be a set of vectors, one per token, where "it" has already absorbed information about "glass", "pitcher" has absorbed information about "poured" and "water", and so on. The input sentence has been fully understood.

The decoder's different job

The decoder's job is fundamentally different. It does not read a sentence; it writes one, word by word. And because of that, it needs two pieces of machinery that the encoder does not:

- It must not cheat by looking at words it has not generated yet.
- It must stay connected to the input sentence it is trying to translate.

These two needs are solved by masked self-attention and cross-attention, respectively.

Masked self-attention: the "no peeking" rule

During training, we show the model the full target sentence so it can learn efficiently in parallel. But at each position in that sentence, the model should only be allowed to look at words that came before it. Otherwise, predicting the next word becomes trivial; the answer is sitting right there.

Masking enforces this rule. When the decoder computes attention at position 3, it is blocked from looking at positions 4, 5, 6, and so on. Analogy: imagine writing a sentence one word at a time, with a sheet of paper covering everything to the right of your pen. You can look back at what you have written, but never forward. This is why decoders are called "autoregressive"; each word is generated based only on the words that came before it.

This same masking mechanism is what modern LLMs use when they generate text one token at a time. It is foundational to everything that comes later in the course.

Cross-attention the bridge between the encoder and the decoder

Self-attention, as we saw, lets tokens in a sentence look at other tokens in the same sentence. Cross-attention is a variation: it lets the decoder look at the encoder's output.

Mechanically, it uses the same query-key-value idea from Section 4, but with a twist. The queries come from the decoder (“what am I trying to generate right now?”), while the keys and values come from the encoder (“what did the input sentence say?”). So as the decoder generates the French translation of our anchor sentence, each token it produces can reach back and attend to the encoder’s representation of the original English.

Analogy: a human translator writing the French version while keeping one finger on the English original, glancing back at the relevant word whenever needed.

Putting the decoder block together

With both pieces in place, the full picture clicks. Every decoder layer performs three operations in sequence:

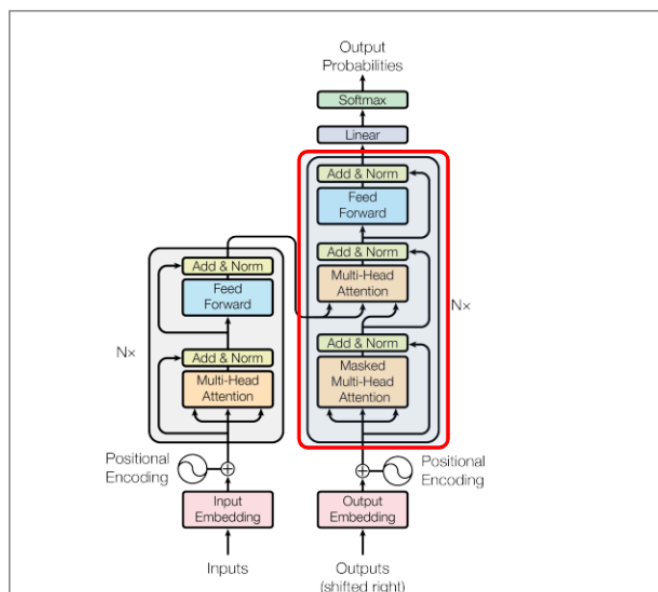
- **Masked self-attention** looks back at what has been generated so far.
- **Cross-attention** looks over the input sentence via the encoder’s output.
- **Feed-forward network** processes the combined information.

Each of these is wrapped with the same plumbing residual connections and layer normalization described in Section 5. And like the encoder, the decoder is a stack of these layers, not just one.

The output layer

At the top of the decoder stack, the final vector for the current position passes through one more small layer that converts it into a probability distribution over the model’s entire vocabulary. In plain terms: the model produces a score for every possible next word, indicating how likely each one is. The model then picks a word from this distribution, typically the most likely one, or a sample from the top candidates, and that word becomes the next token in the generated sentence.

The process then repeats: the newly generated word is fed back in, the decoder runs again, and another word comes out. This continues until the sentence is complete.



Wrapping up

We have now walked through every component of the original Transformer from tokens and embeddings, through positional encoding and attention, past the supporting plumbing, and into the full encoder-decoder assembly with masking and cross-attention. This is the architecture introduced in “Attention Is All You Need,” and it is the foundation on which every modern Large Language Model is built.

Up next, we will see what happens when researchers realized you do not always need both halves and how the family of Transformer variants that grew from this one architecture led directly to the LLMs we use today.